

ESCUELA MILITAR DE INGENIERÍA – EMI

Asignatura: Seminario de Software

Docente: Ing. Jaime Gabriel Soto Gonzales

PRÁCTICA: CONTROL Y SEGUIMIENTO DE UN TRABAJO DE GRADO CON GIT Y GITHUB

Repositorio personal para gestionar las versiones del documento, documentación técnica y código del proyecto

1. Objetivo

Aplicar Git y GitHub como herramientas de control de versiones y seguimiento de un trabajo de grado o tesis, permitiendo al estudiante conservar y demostrar la evolución de su documento académico, documentación técnica y, cuando corresponda, código fuente.

2. Enfoque de la práctica

La práctica será principalmente individual. Cada estudiante creará un repositorio propio y lo utilizará como bitácora de evolución de su trabajo de grado. El elemento central será el documento académico desarrollado en Microsoft Word, acompañado por los archivos técnicos y el código del proyecto cuando corresponda.

El objetivo no es simplemente almacenar archivos en GitHub. Cada modificación importante deberá quedar registrada mediante commits, permitiendo reconstruir la evolución del trabajo.

3. ¿Qué se controlará con Git?

- Documento principal del trabajo de grado (.docx).
- Documentación técnica complementaria.
- Diagramas y material técnico.
- Código fuente del proyecto, cuando corresponda.
- Registro de avances y cambios.
- Versiones importantes mediante Tags o Releases.

Importante: el estudiante puede conservar un mismo archivo Word y registrar sus cambios mediante Git. También puede crear copias versionadas del documento cuando el docente lo considere conveniente.

4. Estructura recomendada del repositorio

```
trabajo-grado-nombre-apellido/  
├── README.md  
├── CHANGELOG.md  
├── .gitignore  
├── documento/  
│   └── Trabajo_Grado.docx  
├── docs/  
│   ├── diagramas/  
│   ├── metodologia/  
│   ├── arquitectura/  
│   └── decisiones/  
└── src/  
    └── código del proyecto
```

5. Actividad 1 – Crear el repositorio personal

Cada estudiante deberá crear un repositorio propio en GitHub. El nombre debe identificar claramente el trabajo.

Ejemplo: trabajo-grado-sistema-gestion

El repositorio podrá ser privado o público según las indicaciones del docente y la naturaleza del trabajo.

6. Actividad 2 – Preparar el documento Word

El estudiante deberá incorporar al repositorio el documento que utiliza para desarrollar su trabajo de grado. No es necesario que el documento esté terminado.

- Generalidades.
- Planteamiento del problema.
- Objetivos.
- Justificación.
- Marco teórico.
- Metodología.
- Diseño o propuesta.
- Resultados.
- Conclusiones.

7. Actividad 3 – Registrar la primera versión

La primera versión del documento deberá representar el estado real del trabajo al comenzar la práctica.

```
git add .
```

```
git commit -m "Agrega versión inicial del trabajo de grado"
```

```
git push origin main
```

8. Actividad 4 – Registrar cada avance mediante commits

Cada vez que el estudiante realice un avance significativo en el documento, deberá guardar los cambios, revisarlos con Git y registrar un commit. Ejm:

- Actualiza planteamiento del problema
- Agrega antecedentes de investigación
- Modifica objetivos específicos
- Actualiza marco teórico
- Corrige metodología según observaciones
- Agrega diseño de arquitectura
- Incorpora resultados de pruebas

9. Actividad 5 – Crear versiones académicas

Además del historial normal de Git, el estudiante deberá identificar hitos importantes de su documento mediante Tags o Releases. Ejm:

Versión	Contenido	Ejemplo
v0.1	Propuesta inicial	Perfil del trabajo
v0.2	Planteamiento y objetivos	Capítulo inicial
v0.3	Marco teórico	Versión revisada

v0.4	Metodología y diseño	Propuesta técnica
v0.5	Resultados preliminares	Primera validación
v1.0	Versión final	Documento completo

10. Actividad 6 – Uso de Issues para las observaciones

Los Issues se utilizarán como registro de tareas pendientes, correcciones u observaciones del trabajo. Ejm:

- Revisar planteamiento del problema.
- Agregar referencias bibliográficas.
- Corregir objetivos.
- Actualizar diagrama de arquitectura.
- Incorporar observaciones del tutor.
- Revisar formato del capítulo 3.

Cuando una observación sea atendida, el estudiante deberá cerrar el Issue y, cuando sea posible, relacionarlo con el commit donde se realizó la corrección.

11. Actividad 7 – Registro de observaciones del tutor

Cuando el tutor realice observaciones sobre el documento, el estudiante deberá registrarlas como Issues. De esta manera, GitHub permitirá demostrar qué observaciones fueron recibidas y cuáles fueron atendidas.

Ejemplo de Issue: "Corregir metodología según observaciones del tutor – reunión 15/08/2026".

12. Actividad 8 – CHANGELOG / Bitácora de avances

El archivo CHANGELOG.md deberá registrar los cambios importantes realizados en el trabajo.

Fecha	Versión	Cambio	Observación
10/08/2026	v0.1	Documento inicial	Creación del repositorio
17/08/2026	v0.2	Actualización de objetivos	Observaciones del tutor

13. Actividad 9 – Colaboración opcional

Aunque el repositorio es individual, el estudiante podrá invitar a un compañero o tutor como colaborador. La colaboración puede utilizarse para revisar documentación, código o cambios específicos.

1. Crear una rama para una modificación.
2. Realizar los cambios.
3. Crear un Pull Request.
4. Solicitar revisión.
5. Atender observaciones.
6. Realizar el merge.

14. Actividad 10 – Demostrar el historial del documento

El estudiante deberá demostrar que puede utilizar GitHub para consultar la evolución de su trabajo.

- Mostrar los commits realizados.
- Identificar qué cambios se realizaron en una fecha determinada.
- Consultar versiones anteriores.
- Mostrar un Tag o Release.

- Mostrar Issues atendidos.
- Relacionar una observación con el cambio realizado.
- Recuperar una versión anterior cuando sea necesario.

15. Código fuente del proyecto

El código fuente deberá incorporarse al mismo repositorio o a un repositorio relacionado. El estudiante deberá utilizar Git para registrar también la evolución del código.

16. Buenas prácticas

7. No utilizar GitHub únicamente como almacenamiento.
8. Realizar commits después de avances significativos.
9. Utilizar mensajes de commit claros.
10. No subir contraseñas, tokens, claves privadas ni información confidencial.
11. Utilizar .gitignore para archivos innecesarios o generados.
12. Mantener actualizado el README.
13. Mantener organizada la documentación.
14. Utilizar repositorio privado cuando el trabajo contenga información restringida.

17. Evidencias a presentar

- Enlace al repositorio personal.
- Documento Word del trabajo de grado.
- README.md.
- Historial de commits.
- Mínimo 5 commits correspondientes a avances reales.
- Mínimo 5 Issues utilizados para tareas u observaciones.
- CHANGELOG.md.
- Al menos un Tag o Release.
- Evidencia de una versión inicial y una versión posterior del documento.
- Evidencia de recuperación o consulta de una versión anterior.
- Pull Request de colaboración, si se realizó.

18. Criterios de evaluación

Criterio	Ponderación
Configuración y organización del repositorio	10%
Control de versiones del documento Word	25%
Calidad y trazabilidad de commits	20%
Uso de Issues y seguimiento de observaciones	15%
CHANGELOG y Tags/Releases	15%
Documentación técnica y código, cuando corresponda	10%
Presentación de evidencias	5%

19. Entregable final

El estudiante entregará el enlace a su repositorio de GitHub. El repositorio deberá permitir observar claramente la evolución del documento y del proyecto durante el periodo de trabajo.